

Practical - 2

Problem Statement: Implement Page Rank Algorithm.

```
In [12]: # Import libraries
import numpy as np
```

```
In [13]: # Constants for PageRank
threshold = 1e-13
beta = 0.85
```

```
In [14]: # Spider Trap Network represented as adjacency matrix
A = [
    [0, 0, 1, 0],
    [1, 0, 0, 0],
    [1, 1, 0, 0],
    [1, 1, 0, 1]
]

# Convert adjacency matrix to a numpy array
arr = np.array(A, dtype=float)
```

```
In [15]: # Calculate summation of columns
s = []
for i in range(len(A)):
    s.append(np.sum(arr[:, i]))

print("Summation of columns: ", s)
```

```
Summation of columns: [3.0, 2.0, 1.0, 1.0]
```

```
In [ ]: # Create the column stochastic probability matrix, M
M = arr.copy()
for j in range(len(A)):
    if s[j] != 0:
        M[:, j] = M[:, j] / s[j]

print("Column stochastic probability matrix, M:")
print(M)
```

```
Column stochastic probability matrix, M:
[[0.          0.          1.          0.          ]
 [0.33333333  0.          0.          0.          ]
 [0.33333333  0.5        0.          0.          ]
 [0.33333333  0.5        0.          1.          ]]
```

```
In [17]: # Initialize rank vector  
r = (1.0 + np.zeros([len(M), 1])) / len(M)  
print("Initial rank vector:")  
print(r)
```

```
Initial rank vector:  
[[0.25]  
 [0.25]  
 [0.25]  
 [0.25]]
```

```
In [18]: # Calculate the uniform rank contribution  
uniformR = (1.0 - beta) * r  
r_prev = r.copy()
```

```
In [19]: # PageRank iterations
for i in range(1, 1001):
    print("Iteration: ", i)
    r = beta * np.matmul(M, r_prev) + uniformR
    print("The rank vector: ")
    print(r)

    diff = np.sum(abs(r - r_prev))
    if diff < threshold:
        break
    r_prev = r.copy()
```

```
Iteration: 1
The rank vector:
[[0.25      ]
 [0.10833333]
 [0.21458333]
 [0.42708333]]
Iteration: 2
The rank vector:
[[0.21989583]
 [0.10833333]
 [0.154375  ]
 [0.51739583]]
Iteration: 3
The rank vector:
[[0.16871875]
 [0.09980382]
 [0.14584549]
 [0.58563194]]
Iteration: 4
The rank vector:
[[0.16146866]
 [0.08530365]
 [0.12772027]
 [0.62550742]]
Iteration: 5
The rank vector:
[[0.14606223]
 [0.08324945]
 [0.1195035  ]
 [0.65118481]]
Iteration: 6
The rank vector:
[[0.13907798]
 [0.0788843  ]
 [0.11426532]
 [0.66777241]]
Iteration: 7
The rank vector:
[[0.13462552]
 [0.07690543]
 [0.11043125]
```

```
[0.6780378 ]]  
Iteration: 8  
The rank vector:  
[[0.13136657]  
 [0.0756439 ]  
 [0.1083287 ]  
 [0.68466083]]  
Iteration: 9  
The rank vector:  
[[0.1295794 ]  
 [0.07472053]  
 [0.10686918]  
 [0.68883089]]  
Iteration: 10  
The rank vector:  
[[0.12833881]  
 [0.07421416]  
 [0.10597039]  
 [0.69147664]]  
Iteration: 11  
The rank vector:  
[[0.12757483]  
 [0.07386266]  
 [0.10540368]  
 [0.69315883]]  
Iteration: 12  
The rank vector:  
[[0.12709313]  
 [0.0736462 ]  
 [0.10503783]  
 [0.69422284]]  
Iteration: 13  
The rank vector:  
[[0.12678216]  
 [0.07350972]  
 [0.10480936]  
 [0.69489877]]  
Iteration: 14  
The rank vector:  
[[0.12658795]  
 [0.07342161]  
 [0.10466324]  
 [0.69532719]]  
Iteration: 15  
The rank vector:  
[[0.12646376]  
 [0.07336659]  
 [0.10457077]  
 [0.69559889]]  
Iteration: 16  
The rank vector:  
[[0.12638516]  
 [0.0733314 ]  
 [0.1045122 ]
```

```
[0.69577125]]
Iteration: 17
The rank vector:
[[0.12633537]
 [0.07330913]
 [0.10447497]
 [0.69588053]]
Iteration: 18
The rank vector:
[[0.12630373]
 [0.07329502]
 [0.1044514 ]
 [0.69594985]]
Iteration: 19
The rank vector:
[[0.12628369]
 [0.07328606]
 [0.10443644]
 [0.69599382]]
Iteration: 20
The rank vector:
[[0.12627097]
 [0.07328038]
 [0.10442695]
 [0.6960217 ]]
Iteration: 21
The rank vector:
[[0.12626291]
 [0.07327678]
 [0.10442094]
 [0.69603938]]
Iteration: 22
The rank vector:
[[0.1262578 ]
 [0.07327449]
 [0.10441712]
 [0.69605059]]
Iteration: 23
The rank vector:
[[0.12625455]
 [0.07327304]
 [0.1044147 ]
 [0.6960577 ]]
Iteration: 24
The rank vector:
[[0.1262525 ]
 [0.07327212]
 [0.10441317]
 [0.69606221]]
Iteration: 25
The rank vector:
[[0.12625119]
 [0.07327154]
 [0.10441219]
```

```
[0.69606508]]
Iteration: 26
The rank vector:
[[0.12625036]
 [0.07327117]
 [0.10441158]
 [0.69606689]]
Iteration: 27
The rank vector:
[[0.12624984]
 [0.07327094]
 [0.10441118]
 [0.69606804]]
Iteration: 28
The rank vector:
[[0.12624951]
 [0.07327079]
 [0.10441094]
 [0.69606877]]
Iteration: 29
The rank vector:
[[0.1262493 ]
 [0.07327069]
 [0.10441078]
 [0.69606923]]
Iteration: 30
The rank vector:
[[0.12624916]
 [0.07327063]
 [0.10441068]
 [0.69606953]]
Iteration: 31
The rank vector:
[[0.12624908]
 [0.0732706 ]
 [0.10441062]
 [0.69606971]]
Iteration: 32
The rank vector:
[[0.12624902]
 [0.07327057]
 [0.10441057]
 [0.69606983]]
Iteration: 33
The rank vector:
[[0.12624899]
 [0.07327056]
 [0.10441055]
 [0.69606991]]
Iteration: 34
The rank vector:
[[0.12624897]
 [0.07327055]
 [0.10441053]
```

```
[0.69606995]]
Iteration: 35
The rank vector:
[[0.12624895]
 [0.07327054]
 [0.10441052]
 [0.69606998]]
Iteration: 36
The rank vector:
[[0.12624894]
 [0.07327054]
 [0.10441052]
 [0.69607   ]]
Iteration: 37
The rank vector:
[[0.12624894]
 [0.07327053]
 [0.10441051]
 [0.69607001]]
Iteration: 38
The rank vector:
[[0.12624894]
 [0.07327053]
 [0.10441051]
 [0.69607002]]
Iteration: 39
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 40
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 41
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 42
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 43
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
```

```
[0.69607003]]
Iteration: 44
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 45
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 46
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 47
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607003]]
Iteration: 48
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 49
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 50
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 51
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 52
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
```

```
[0.69607004]]
Iteration: 53
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 54
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 55
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 56
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 57
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 58
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 59
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 60
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 61
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
```

```
[0.69607004]]
Iteration: 62
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 63
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 64
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
Iteration: 65
The rank vector:
[[0.12624893]
 [0.07327053]
 [0.10441051]
 [0.69607004]]
```

```
In [20]: # Display the final rank vector
print("The final rank vector: ")
print(r[:, 0])
```

```
The final rank vector:
[0.12624893 0.07327053 0.10441051 0.69607004]
```
